NodeJS

Importance

- Node.js is an opensource and crossplatform JavaScript runtime environment.
- *A Node.js app runs in a single process, without creating a new thread for every request.* Node.js provides a set of asynchronous I/O primitives in its standard library that prevent JavaScript code from blocking.
- Libraries in Node.js are written using nonblocking paradigms, making blocking behaviour the exception rather than the norm.
- When Node.js performs an I/O operation, like reading from the network, accessing a database or the filesystem, instead of blocking the thread and wasting CPU cycles waiting, Node.js will resume the operations when the response comes back.

Synchronous Code

SYNCRONOUS CODE

```
let fs = require('fs');

//Reading file synchronously
let textIn = fs.readFileSync('./txt/input.txt', 'utf-8');
console.log(textIn);
```

OUTPUT

SINGLE THDEAD

```
let fs = require('fs');
```

NODE JS RUNTIME

SYNCRONOUS CODE

```
let fs = require('fs');

//Reading file synchronously
let textIn = fs.readFileSync('./txt/input.txt', 'utf-8');
console.log(textIn);
```

OUTPUT

SINGLE THDEAD

fs.readFileSy BLOCKED t', 'utf-8');

NODE JS RUNTIME

SYNCRONOUS CODE

```
let fs = require('fs');

//Reading file synchronously
let textIn = fs.readFileSync('./txt/input.txt', 'utf-8');
console.log(textIn);
```

This is the file content of input.txt

OUTPUT

SINGLE THDEAD

NODE JS RUNTIME

ASynchronous Code

ASYNCRONOUS CODE

```
let fs = require('fs');

//Reading file synchronously
fs.readFile('./txt/input.txt', 'utf-8', (err, data) =>{
    console.log(data);
});
console.log('Reading file...');
```

OUTPUT

SINGLE THDEAD

```
let fs = require('fs');
```

NODE JS RUNTIME

ASYNCRONOUS CODE

```
let fs = require('fs');

//Reading file synchronously
fs.readFile('./txt/input.txt', 'utf-8', (err, data) =>{
    console.log(data);
});
console.log('Reading file...');
```

Reading file...

OUTPUT

SINGLE THDEAD

BACKGROUND

```
fs.readFile('./txt/input.txt', 'utf-8', (err, data) =>{
    console.log(data);
});
```

NODE JS RUNTIME

ASYNCRONOUS CODE

```
let fs = require('fs');

//Reading file synchronously
fs.readFile('./txt/input.txt', 'utf-8', (err, data) =>{
    console.log(data);
});
console.log('Reading file...');
```

Reading file...

OUTPUT

SINGLE THDEAD

```
(err, data) => {
    console.log(data);
}
```

BACKGROUND

NODE JS RUNTIME

ASYNCRONOUS CODE

```
let fs = require('fs');

//Reading file synchronously
fs.readFile('./txt/input.txt', 'utf-8', (err, data) =>{
    console.log(data);
});
console.log('Reading file...');
```

Reading file...

This is the file content of input.txt

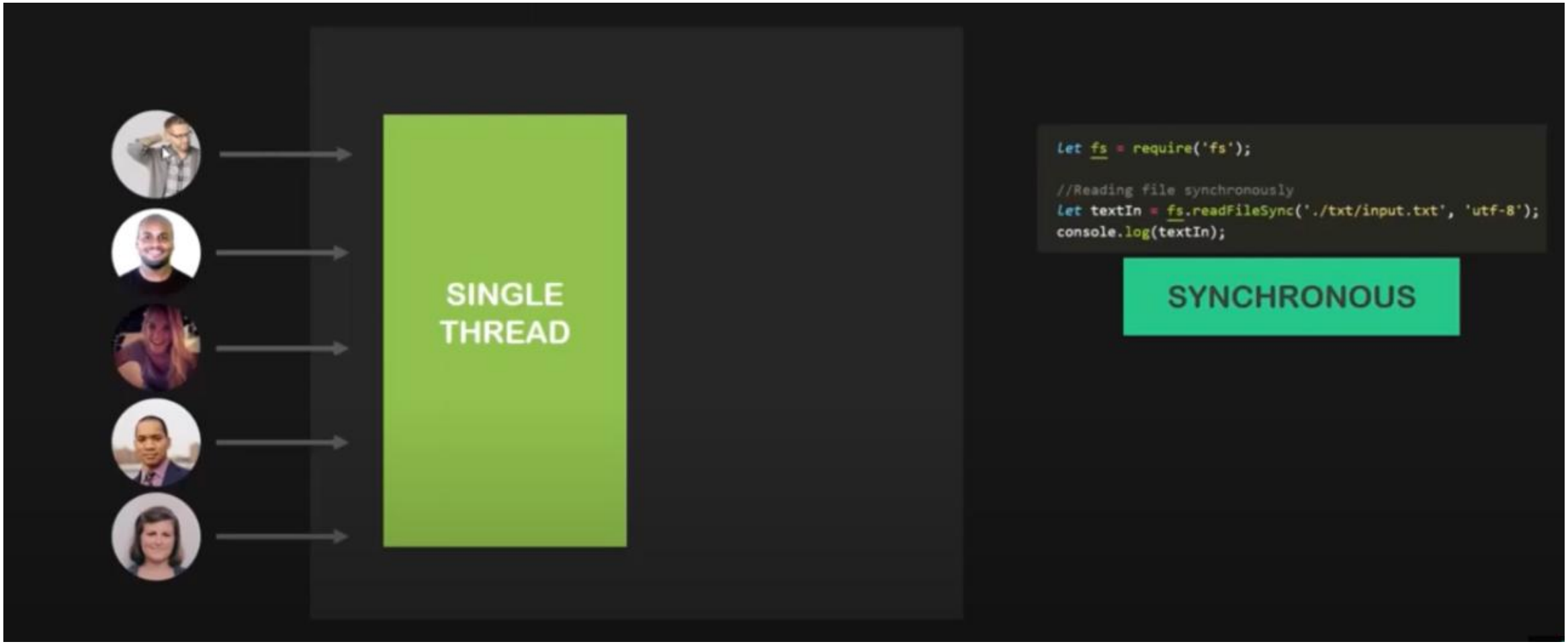
OUTPUT

SINGLE THDEAD

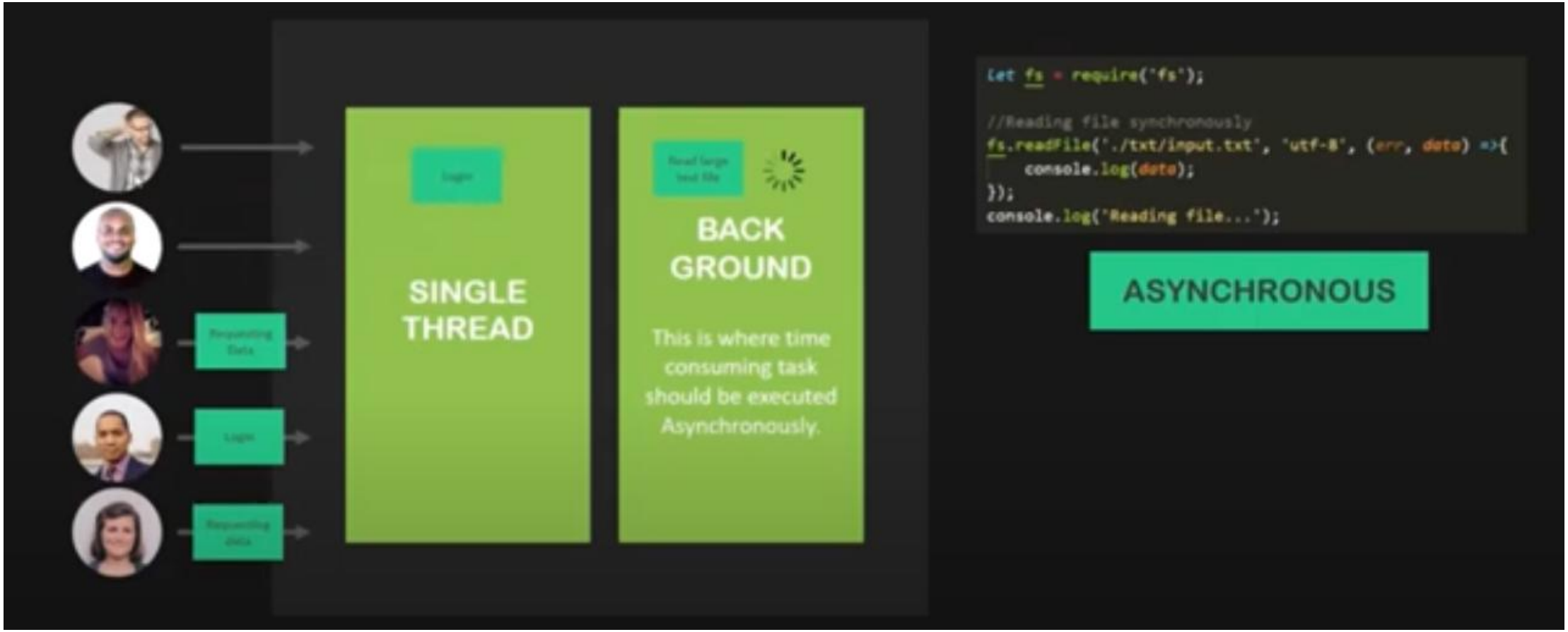
BACKGROUND

NODE JS RUNTIME

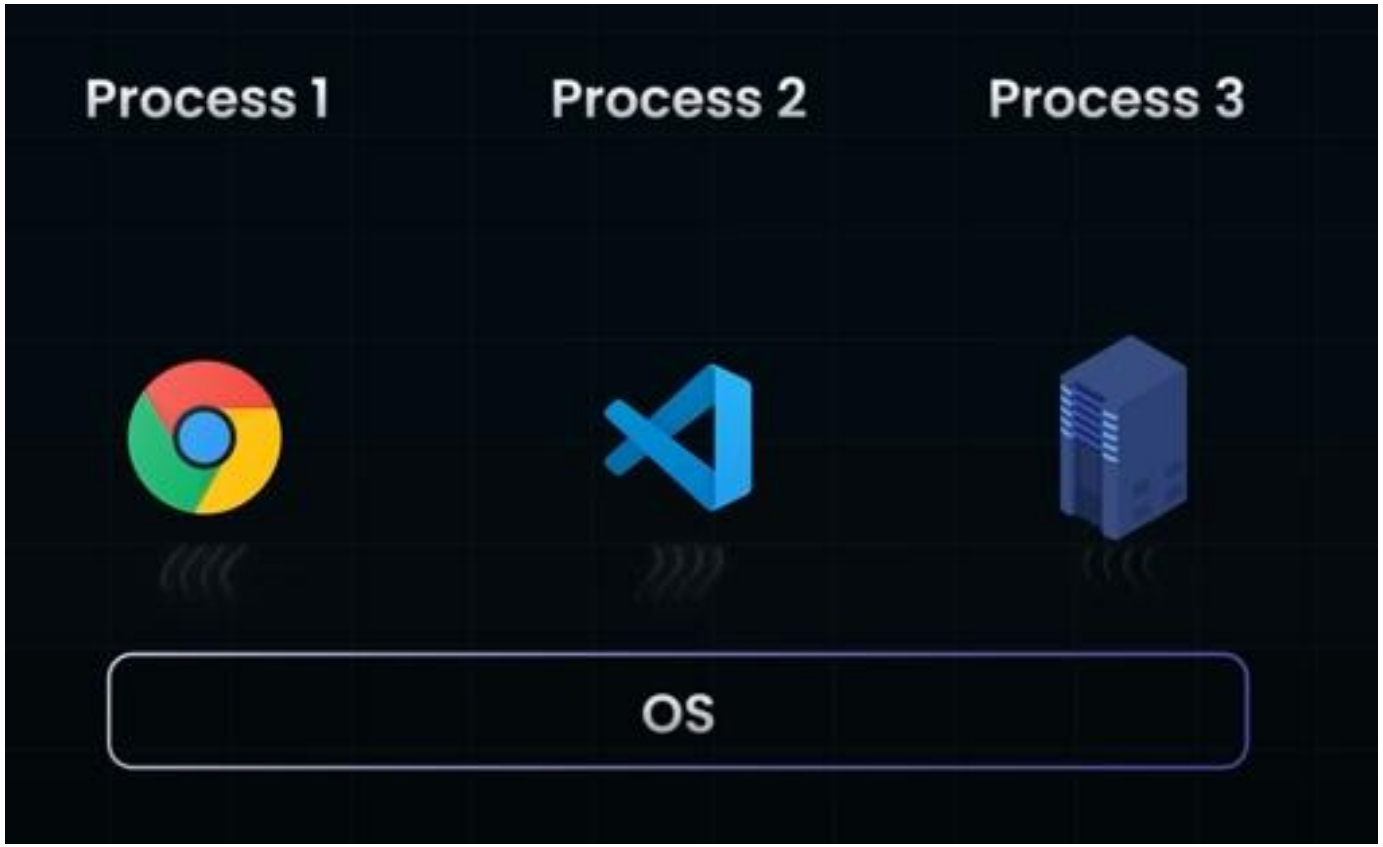
Why Asynchronous?



Why Asynchronous?

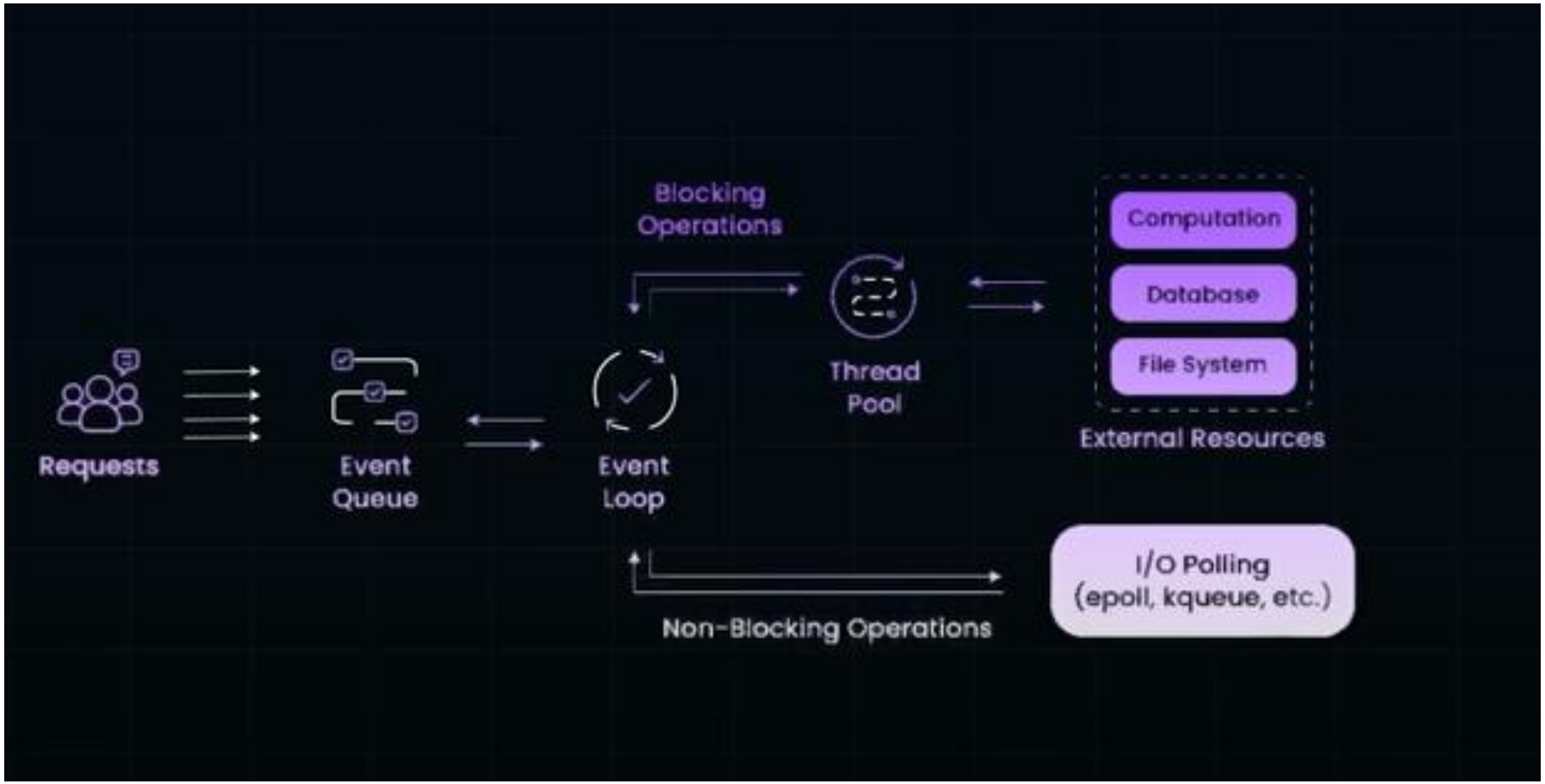


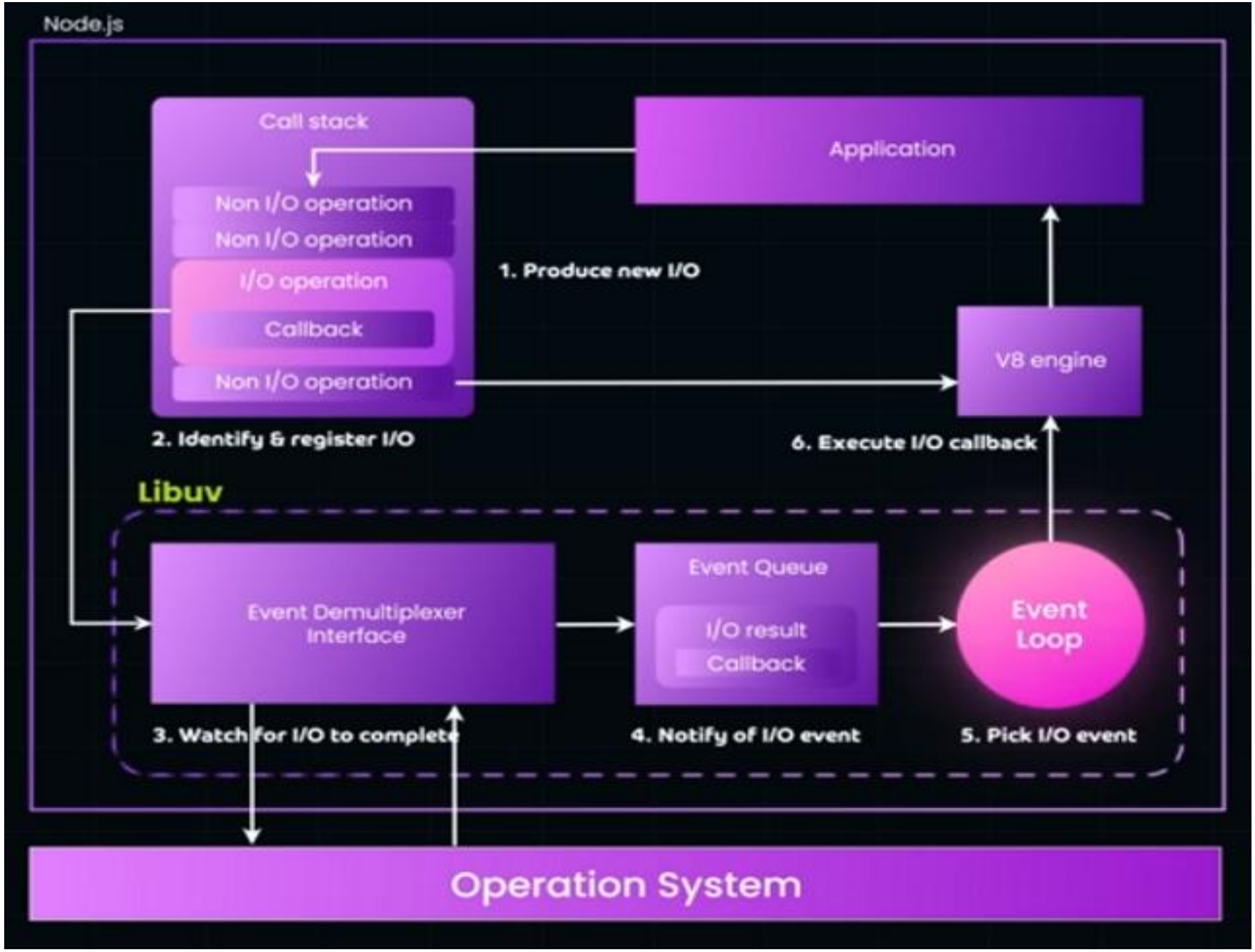
What is a Process?



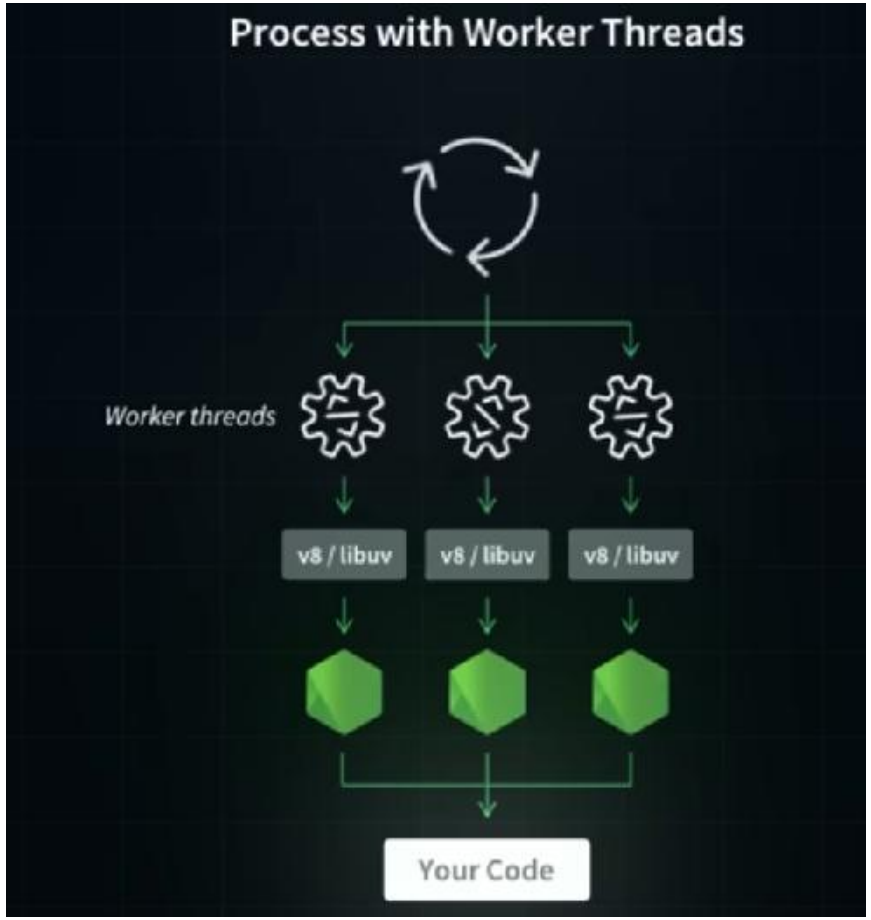
Single Threaded Problem

Architecture of Node




Node.js
Call stack
Non I/O operation
Non I/O operation
I/O operation
Callback
Non I/O operation
Application
1. Produce new I/O
V8 engine
2. Identify & register I/O
6. Execute I/O callback
Libuv
Event Demultiplexer Interface
Event Queue
I/O result
Callback
Event Loop
3. Watch for I/O to complete
4. Notify of I/O event
5. Pick I/O event
Operation System

CPU Intensive Task are handled by Worker Thread



Importance

- Node.js has a unique advantage because millions of frontend developers that write JavaScript for the browser are now able to write the serverside code in addition to the client side code without the need to learn a completely different language.

- In Node.js the new ECMAScript standards can be used without problems, as you don't have to wait for all your users to update their browsers you are in charge of deciding which ECMAScript version to use by changing the Node.js version, and you can also enable specific experimental features by running Node.js with flags.

- Node.js is one of the popular choices for developing RESTful APIs, microservices and web applications.

Features of Node.js

1. **Extremely fast:** Node.js is built on Google Chrome's V8 JavaScript Engine, so its library is very fast in code execution.
2. **I/O is Asynchronous and Event Driven:** All APIs of Node.js library are asynchronous i.e. nonblocking. So a Node.js based server never waits for an API to return data. The server moves to the next API after calling it and a notification mechanism of Events of Node.js helps the server to get a response from the previous API call. It is also a reason that it is very fast.
3. **Single threaded:** Node.js follows a single threaded model with event looping.
4. **Highly Scalable:** Node.js is highly scalable because event mechanism helps the server to respond in a nonblocking way.
5. **No buffering:** Node.js cuts down the overall processing time while uploading audio and video files. Node.js applications never buffer any data. These applications simply output the data in chunks.
6. **Open source:** Node.js has an open source community which has produced many excellent modules to add additional capabilities to Node.js applications.
7. **License:** Node.js is released under the MIT license.

What Can Node.js Do?

Node.js is a runtime environment that allows you to run JavaScript on the serverside. Here's a breakdown:

1. **Generate dynamic page content:** Yes, Node.js can dynamically generate HTML content on the serverside, allowing for dynamic web page generation.
2. **File manipulation**: Node.js has builtin modules that allow you to interact with the file system, including creating, opening, reading, writing, deleting, and closing files on the server.
3. **Collect form data:** Node.js can handle form submissions from web pages. It can parse form data and process it on the server.
4. **Database operations:** Node.js can interact with databases, both SQL and NoSQL. This includes adding, deleting, and modifying data in databases.

   These functionalities are often used to create web applications, APIs, and other serverside applications.

Comparing File Handling in PHP/ASP and Node.js

PHP/ASP File Handling:

1. Sends task to file system.
2. Waits for file system to open and read the file.
3. Returns content to client.
4. Ready to handle the next request.

Node.js Advantages:

Eliminates waiting, continues with next request.
Singlethreaded, nonblocking, asynchronous.
Memory efficient.

Node.js File Handling:

1. Sends task to file system.
2. Ready to handle the next request.
3. When file system completes operation, returns content to client.

First Program in Node js to create server

File Edit Selection View Go Run Terminal Help

Examples_lec

EXPLORER

EXAMPLES_LEC

- php-8.4.3
- disp.js
- disp1.js
- exampmodal.html
- firstprg.html
- fonttag.html
- form1.html
- funcexamp.js
- jqueryappend.html
- jquerynumver.html
- jquerystyle.html
- learn.php
- learnb.html
- learnhyperlink.html
- new.js
- package.json 1
- phpfirst.html
- phpstr.html
- samplebutton.html
- scriptfirst.js
- str2php.html
- trialcode.php
- unorderlist.html
- usehrtag.html

OUTLINE

TIMELINE

p | phpfirst.html | trialcode.php | samplebutton.html | jquerystyle.html | jquery

{} package.json > ...

```
1  {
2    "name": "examples_lec",
3    "version": "1.0.0",
4    "description": "my first app",
5    "main": "disp1.js",
     ▷Debug
6    "scripts": {
7      "test": "echo \"Error: no test specified\" && exit 1"
8    },
9    "author": "",
10   "license": "ISC"
11 }
12
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Microsoft Windows [Version 10.0.22631.4751]
(c) Microsoft Corporation. All rights reserved.

D:\AI&DS_2024-25 Data\Web Technology\Examples_lec>npm init
This utility will walk you through creating a package.json file.
It only covers the most common items, and tries to guess sensible defaults.

See `npm help init` for definitive documentation on these fields
and exactly what they do.

Use `npm install <pkg>` afterwards to install a package and
save it as a dependency in the package.json file.

Press ^C at any time to quit.
package name: (examples_lec)
version: (1.0.0)
description: my first app
entry point: (disp.js)
```

File Edit Selection View Go Run Terminal Help

Examples_lec

EXPLORER

EXAMPLES_LEC

- php-8.4.3
- disp.js
- disp1.js
- exampmodal.html
- firstprg.html
- fonttag.html
- form1.html
- funcexamp.js
- jqueryappend.html
- jquerynumver.html
- jquerystyle.html
- learn.php
- learnb.html
- learnhyperlink.html
- new.js
- package.json 1
- phpfirst.html
- phpstr.html
- samplebutton.html
- scriptfirst.js
- str2php.html
- trialcode.php
- unorderlist.html
- usehrtag.html

OUTLINE

p | phpfirst.html | trialcode.php | samplebutton.html | jquerystyle.html | jquerynumver.html

disp1.js > ...

```javascript
// Import the http module
const http = require('node:http');

// Define the hostname and port
const hostname = '127.0.0.1';
const port = 3000;

// Create the server
const server = http.createServer((req, res) => {
res.statusCode = 200;
res.setHeader('Content-Type', 'text/plain');
res.end('Hello World\n');
});

// Start the server
server.listen(port, hostname, () => {
console.log(`Server running at http://${hostname}:${port}/`);
});
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Press ^C at any time to quit.
package name: (examples_lec)
version: (1.0.0)
description: my first app
entry point: (disp.js)
test command:
git repository:
keywords:
author:
license: (ISC)
About to write to D:\AI&DS_2024-25 Data\Web Technology\Examples_lec\package.json:

{
  "name": "examples_lec",
  "version": "1.0.0",
  "description": "my first app",
```

File Edit Selection View Go Run Terminal Help

Examples_lec

EXPLORER

EXAMPLES_LEC

- php-8.4.3
- disp.js
- disp1.js
- exampmodal.html
- firstprg.html
- fonttag.html
- form1.html
- funcexamp.js
- jqueryappend.html
- jquerynumver.html
- jquerystyle.html
- learn.php
- learnb.html
- learnhyperlink.html
- new.js
- package.json 1
- phpfirst.html
- phpstr.html
- samplebutton.html
- scriptfirst.js
- str2php.html
- trialcode.php
- unorderlist.html
- usehrtag.html

> OUTLINE

> TIMELINE

p | phpfirst.html | trialcode.php | samplebutton.html | jquerystyle.html

disp1.js > ...

```javascript
// Import the http module
const http = require('node:http');

// Define the hostname and port
const hostname = '127.0.0.1';
const port = 3000;

// Create the server
const server = http.createServer((req, res) => {
res.statusCode = 200;
res.setHeader('Content-Type', 'text/plain');
res.end('Hello World\n');
});

// Start the server
server.listen(port, hostname, () => {
console.log(`Server running at http://${hostname}:${port}/`);
});
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
{
  "name": "examples_lec",
  "version": "1.0.0",
  "description": "my first app",
  "main": "disp.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "",
  "license": "ISC"
}


Is this OK? (yes) yes

D:\AI&DS_2024-25 Data\Web Technology\Examples_lec>node disp1.js
Server running at http://127.0.0.1:3000/
```

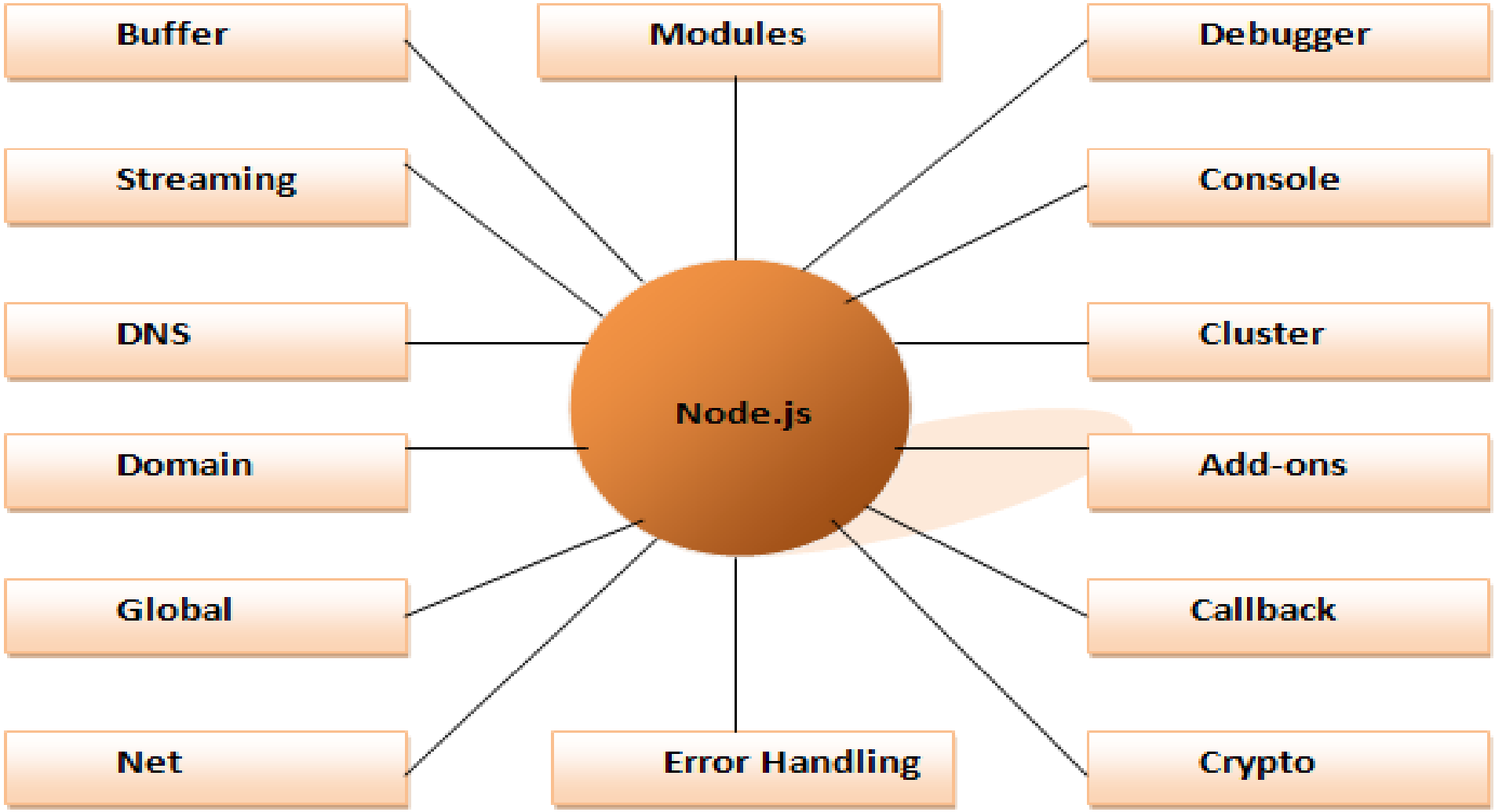
node js code to createserver - Sea

127.0.0.1:3000

127.0.0.1:3000

Hello World

Different parts of Node.js



How to install NodeJS

Creating a Consolebased Node Application

Step 1: Set Up Your Environment

- Install Node.js: Download and install Node.js from nodejs.org.
- Verify Installation: Open your terminal or command prompt and run `node v` to check if Node.js is installed.

Step 2: Initialize Your Node Project

- Create a New Directory: Create a new directory for your project using `mkdir mynodeapp`.
- Navigate to the Directory: Move into your project directory using `cd mynodeapp`.
- Initialize Node Project: Run `npm init y` to initialize a new Node.js project with default settings.

Step 3: Write Your Node.js Script

- Create a JavaScript File: Create a new JavaScript file, e.g., `app.js`, in your project directory.
- Write Your Code: Write your Node.js script in `app.js`, including any consolebased functionality you want to implement.

Creating a Consolebased Node Application

Step 4: Run Your Node Application

- Execute Your Script: In the terminal, run `node app.js` to execute your Node.js application.
- Check Output: Check the terminal for any console.log statements or output generated by your script.

Step 5: Iterate and Improve

- Test and Debug: Test your application thoroughly and debug any errors or issues.
- Refine Your Code: Refactor and improve your code as needed, adding new features or functionality.

Step 6: Share Your Application

- Version Control: Consider using Git for version control to track changes to your codebase.
- Share Your Project: Share your Node.js project with others on platforms like GitHub for collaboration or feedback.

Creating a Web-based Node Application

Step 1: Set Up Your Environment

- Install Node.js: Download and install Node.js from nodejs.org.
- Verify Installation: Open your terminal or command prompt and run `node v` to check if Node.js is installed.

Step 2: Initialize Your Node Project

- Create a New Directory: Create a new directory for your project using `mkdir mynodeapp`.
- Navigate to the Directory: Move into your project directory using `cd mynodeapp`.
- Initialize Node Project: Run `npm init y` to initialize a new Node.js project with default settings.
- **Step 3:** Install Express.js
- Install Express.js: Use npm to install Express.js by running `npm install express`.Express.js: A web application framework for Node.js, simplifying the process of building web apps and APIs.

Creating a Web-based Node Application

Step 4: Write Your Node.js Script

- Create a JavaScript File: Create a new JavaScript file, e.g., `app.js`, in your project directory.
- Write Your Code: Write your Node.js script in `app.js`, including setting up routes, handling requests, and defining responses.

Step 5: Create Your HTML Views

- Create Views Directory: Create a directory named `views` to store your HTML files.
- Write HTML Files: Create HTML files for different views/pages of your web application, such as `index.html`, `about.html`, etc.

Step 6: Serve Static Files

- Serve Static Files: Use Express.js to serve static files (e.g., CSS, images) by using the `express.static` middleware.
- Organize Your Files: Place static files (e.g., CSS, images) in a directory named `public` for better organization.

Creating a Web-based Node Application

Step 7: Run Your Node Application

- Execute Your Script: In the terminal, run `node app.js` to start your Node.js application.
- Access Your Web App: Open your web browser and navigate to `http://localhost:3000` (or another port if specified) to view your web application.

Step 8: Iterate and Improve

- Test and Debug: Test your application thoroughly and debug any errors or issues.
- Refine Your Code: Refactor and improve your code as needed, adding new features or functionality.

Step 9: Deployment

- Choose a Hosting Provider: Select a hosting provider (e.g., Heroku, AWS, DigitalOcean) to deploy your Node.js application.
- Deploy Your App: Follow the hosting provider's instructions to deploy your web-based Node.js application.

Modules in Node.js

1. What is a Module?

- A reusable piece of JavaScript code that encapsulates specific functionality.
- Helps organize code into logical units, making it easier to manage and maintain large applications.

2. Encapsulation and Modularity

- Each module in Node.js encapsulates its own variables, functions, and classes, keeping them private by default.
- Node.js follows the CommonJS module system, treating each file as a separate module.

3. Exporting Functionality

- Modules can export functionalities using `module.exports` or `exports`.
- Exported functionalities are made accessible to other modules through `require()`.

Modules in Node.js

4. Importing Modules

- Modules are imported into other modules using the `require()` function.
- When a module is required, its code is executed, and its exports are returned for use.

5. Types of Modules

- Built-in Modules: Provided by Node.js, e.g., `fs`, `http`, `path`.
- User-defined Modules: Created by developers to encapsulate specific functionalities.

6. Benefits of Modules

- Facilitate code organization and modularity.
- Promote code reuse and maintainability.
- Essential for building scalable and maintainable applications in Node.js.

Node.js HTTP Module

1. Introduction to HTTP Module

- Node.js includes a built-in HTTP module that allows you to create HTTP servers and make HTTP requests.
- Provides a set of functions and classes for handling HTTP-related tasks.

2. Creating an HTTP Server

- Use the `http.createServer()` method to create an HTTP server instance.
- Define request/response handlers using callbacks or event listeners.

3. Handling HTTP Requests

- Access request details such as URL, method, headers, and body.
- Process incoming requests and generate appropriate responses.

Node.js HTTP Module

4. Sending HTTP Responses

Use the `response.writeHead()` method to set the status code and headers of the response.

Write the response body using the `response.write()` method.

End the response with `response.end()`.

Example:

```
const http = require('http');
  const server = http.createServer((req, res) => {
      res.writeHead(200, { 'Content-Type': 'text/plain' });
      res.end(‘Viomesh Sir:NodeJS Class');
  });
  server.listen(3000, () => {
      console.log('Server running on port 3000');
  });
```

Node.js HTTP Module

5. Additional Functionality

- Supports HTTPS for secure communication.
- Can be extended with middleware frameworks like Express.js for more advanced routing and request handling.

6. Benefits of HTTP Module

- Lightweight and efficient for building simple HTTP servers.
- Ideal for handling basic web applications, APIs, and microservices.

Node.js File System Module

1.Introduction to File System Module

- Node.js includes a built-in File System module (`fs`) for interacting with the file system on your computer.

2.Key Functions

- Reading Files: Use `fs.readFile()` or `fs.readFileSync()` to read the contents of a file asynchronously or synchronously, respectively.
- Writing Files: Use `fs.writeFile()` or `fs.writeFileSync()` to write data to a file asynchronously or synchronously, respectively.
- Working with Directories: Functions like `fs.readdir()` and `fs.mkdir()` allow you to work with directories, listing contents or creating new ones.

Node.js File System Module

3.Asynchronous vs. Synchronous Methods

- Asynchronous methods are non-blocking and return immediately, allowing your code to continue executing.
- Synchronous methods block the execution of code until the operation is completed, potentially leading to performance issues in I/O-heavy applications.

Read File

The fs.readFile() method is used to read files on your computer.

Assume we have the following HTML file (located in the same folder as Node.js):

```
<html>
<body>
<h1>My Header</h1>
<p>My paragraph.</p>
</body>
</html>
```

Node.js file that reads the HTML file, and return the content

```
var http = require('http');
var fs = require('fs');
http.createServer(function (req, res) {
  fs.readFile('demofile1.html', function(err, data) {
    res.writeHead(200, {'Content-Type': 'text/html'});
    res.write(data);
    return res.end();
  });
}).listen(8080);
```

Save the code above in a file called "demo_readfile.js", and initiate the file:

C:\Users*Your Name*>node demo_readfile.js

Create File

The File System module has methods for creating new files:

fs.appendFile()

fs.open()

fs.writeFile()

The fs.appendFile() method appends specified content to a file. If the file does not exist, the file will be created:

Example

```
var fs = require('fs');

fs.appendFile('mynewfile1.txt', 'Hello content!', function (err) {
  if (err) throw err;
  console.log('Saved!');
});
```

The fs.open() method takes a "flag" as the second argument, if the flag is "w" for "writing", the specified file is opened for writing. If the file does not exist, an empty file is created:

Create a new, empty file using the open() method:Example

```
var fs = require('fs');

fs.open('mynewfile2.txt', 'w', function (err, file) {
  if (err) throw err;
  console.log('Saved!');
});
```

The fs.writeFile() method replaces the specified file and content if it exists. If the file does not exist, a new file, containing the specified content, will be created:

Create a new file using the writeFile() method:

```
var fs = require('fs');

fs.open('mynewfile2.txt', 'w', function (err, file) {
  if (err) throw err;
  console.log('Saved!');
});
```

Self Study

File System Module

Update Files

Delete Files

Rename Files

Upload Files

NodeJS NPM Package

Node.js with MySQL

Advantages of Using Node.js with MySQL

Using Node.js with MySQL offers several advantages for building web applications:

1.JavaScript Everywhere: With Node.js, you can use JavaScript on both the client and server sides of your application. This allows for consistency in development and easier code sharing between different parts of your application.

2.Asynchronous I/O: Node.js is known for its non-blocking, asynchronous I/O model, which allows it to handle multiple concurrent operations efficiently. This aligns well with MySQL's I/O-bound nature, making Node.js a good choice for interacting with MySQL databases.

3.Performance: Node.js's event-driven architecture and asynchronous nature make it well-suited for I/O-heavy tasks like database operations. It can handle a large number of concurrent database queries without blocking, leading to improved performance and responsiveness in web applications.

Advantages of Using Node.js with MySQL

4.Scalability: Both Node.js and MySQL are known for their scalability. Node.js can handle a large number of concurrent connections efficiently, and MySQL can scale horizontally by adding more servers to distribute the load. Together, they form a scalable stack for building high-performance web applications.

5.Community and Ecosystem: Both Node.js and MySQL have large and active communities, with extensive libraries, frameworks, and tools available to streamline development. This rich ecosystem provides developers with the resources they need to build robust and feature-rich web applications.

6.Ease of Development: Node.js's simple and lightweight runtime, combined with its rich ecosystem of modules, makes it easy to develop and maintain web applications. Integrating Node.js with MySQL allows developers to build data-driven applications quickly and efficiently.

Overall, using Node.js with MySQL provides a powerful and efficient stack for building modern web applications, combining the flexibility and scalability of Node.js with the reliability and performance of MySQL databases.

Advantages of Using Node.js with MySQL

4.Scalability: Both Node.js and MySQL are known for their scalability. Node.js can handle a large number of concurrent connections efficiently, and MySQL can scale horizontally by adding more servers to distribute the load. Together, they form a scalable stack for building high-performance web applications.

5.Community and Ecosystem: Both Node.js and MySQL have large and active communities, with extensive libraries, frameworks, and tools available to streamline development. This rich ecosystem provides developers with the resources they need to build robust and feature-rich web applications.

6.Ease of Development: Node.js's simple and lightweight runtime, combined with its rich ecosystem of modules, makes it easy to develop and maintain web applications. Integrating Node.js with MySQL allows developers to build data-driven applications quickly and efficiently.

Overall, using Node.js with MySQL provides a powerful and efficient stack for building modern web applications, combining the flexibility and scalability of Node.js with the reliability and performance of MySQL databases.

Node.js with MySQL

Introduction to MySQL

- MySQL is a popular open-source relational database management system (RDBMS) known for its reliability, scalability, and performance.
- It's widely used in web development for storing and managing structured data.

Node.js and MySQL

- Node.js can be used to interact with MySQL databases, allowing developers to build powerful and data-driven applications.
- The `mysql` package is commonly used in Node.js applications for MySQL database connectivity.

Node.js with MySQL

Connecting to MySQL Database

- Install Node.js: Download and install Node.js from the official website https://nodejs.org/en/download. This will also install npm (Node Package Manager).

- Install MySQL Driver: Use npm to install a driver that lets Node.js communicate with MySQL. A popular option is

```
npm install mysql
```

Connecting to MySQL:

- Node.js scripts use the installed driver to connect to your MySQL database.

  mysql.createConnection().

Example

```
const mysql = require('mysql');
const connection = mysql.createConnection({
  host: 'localhost',
  user: 'your_username',
  password: 'your_password',
  database: 'your_database'
});
connection.connect(function(err) {
  if (err) throw err;
  console.log("Connected to the database!");
});
```

Node.js with MySQL

Working with MySQL:

- Once connected, you can perform CRUD (Create, Read, Update, Delete) operations on your database. You can write queries to:
- Create tables and insert data
- Fetch data from tables
- Update existing data
- Delete data

The mysql driver provides functions like connection.query() to execute SQL statements and handle results.

Node.js with MySQL

Perform SQL queries: Use the connection object to execute SQL queries using methods like `connection.query()`.

Example: Executing SQL Query

```
connection.query('SELECT * FROM users', (err, rows) => {
    if (err) throw err;
    console.log('Data received from MySQL:');
    console.log(rows);
  });
```